队伍编号	MC2403042
题号	C

基于ARIMA和线性规划的物流网络货量预测与人员排班优化研究

摘要

本文针对电商物流网络提出了对于未来30天各个分拣中心的货量预测以及人员排班问题,同时对于以上问题我们提出了一系列优化模型和求解方法。首先在货量预测问题方面,文章提出了基于ARIMA的预测模型,针对历史货量数据和分拣中心间的关系来实现对未来 30 天各分拣中心每日和每小时货量的准确预测。另外在人员分配问题上，我们使用了线性规划的算法，通过对约束函数和目标函数进行求解，得出一套最优的人员排班方案。

针对问题 1,这是一个**时间序列预测问题**，首先我们对附件1进行**数据预处理**。采用**箱线图**分析来查找异常值，检验到异常值存在后直接删除该异常值，再者对于数据空值我们使用**拉格朗日插值法**填充缺失数据。然后，我们应用**ARIMA模型**进行预测，同时评估给定数据的线性关系。在非线性情况下，我们进行d次差分操作直至达到线性关系。通过确定参数p和q，然后通过ARIMA模型得出未来30天的预测结果计入表1。利用附录2中每小时货量比例的数据，将ARIMA预测结果按比例缩放，最后准确预测每小时货量计入表2。

针对问题 2，这是一个**图论与网络的分析模型**，我们引入图论和网络分析的方法来优化先前分析的预测，以了解各个分拣中心之间的连接和货量流动的关系。引入**节点强度**来估算每个分拣中心的平均货量流动。路径分析可评估物流网络中分拣中心之间的主要货量的流动通道。通过确定每个节点的货量强度，我们对先前的预测进行修正，并得出本问题的预测结果计入表3。最后，基于更新后的预测结果和先前分析，我们得出的每小时货量权重进行小时级的预测计入表4。

针对问题 3，这是一个**线性规划**算法来解决人力资源调度的问题。我们将时间划分为6个区间，并考虑了正式工人数、临时工人数和效率偏差等**决策变量**。通过分析每天的小时货量，我们总结了几个**约束条件**，如正式工每天最多出勤一个班次、人员非负性、正式工人数不超过60人，总工作量不低于预测货量。**目标函数**旨在最小化总人数和效率偏差，同时确保工作量和效率的平衡。通过建立线性规划模型来优化**电商物流网络**，以提高工作效率和增加完成包裹的数量。

针对问题 4，基于问题三，我们**延续线性规划方法**，我们增加了新的**约束条件**：要求每名正式员工的出勤率不得高于百分之85；连续工作天数不得超过7天，即正式工每工作5天休息1天，达到出勤率为0.83、正式工人数不超过200人。**决策变量**包括正式工的班次、临时工的班次以及正式工的休息日安排。**目标函数**旨在最小化正式工的班次和休息日数量。通过建立详细的线性规划模型，最后，我们成功解决了SC60的员工安排问题，确保了生产效率和员工福利的平衡。

关键词: **ARIMA时间序列分析**　　**线性规划**　　**电商物流网络**　　**网络物流模型构建**

目录

1 一、问题重述 1

1.1 问题背景 1

1.2 问题重述 1

2 二、问题分析 1

2.1 问题一的分析 1

2.2 问题二的分析 2

2.3 问题三的分析 2

2.4 问题四的分析 3

3 三、模型假设 3

4 四、符号说明 4

5 五、模型的建立与求解 4

5.1 问题一： 4

5.1.1 问题一中对未来30天每天的货量预测： 4

5.1.2 问题一中对未来30天每小时的货量预测： 8

5.2 问题二： 9

5.2.1 问题二中对未来30天每天的货量预测： 9

5.2.2 问题二中对未来30天每小时的货量预测： 11

5.3 问题三： 12

5.4 问题四： 15

6 六、模型的评价与推广 17

6.1 模型的优点： 17

6.2 模型的缺点： 18

6.3 模型的推广： 18

7 附录 19

一、问题重述

1.1 问题背景

在订单履约中，电商物流网络由多个环节组成。分拣中心是网络的中间环节。它将包裹发往各个相应的地点，使包裹最后到达消费者手中。整体网络的履约效率和运作成本都与分拣中心管理效率的提升密切相关。

1.2 问题重述

在问题一中，我们进行货量预测模型构建，然后对物流网络中的57个分拣中心进行两个方面的货量预测，先是未来30天的时间维度，再是基于此的每个小时预测，将预测结果写入结果表中。

在问题二中，基于过去90天的每个分拣中心之间的平均运输量和57个分拣中心未来30天的运输线路之间变化（增加或者减少），进行合理的预测,最后将预测结果写入结果表中。

在问题三中，基于问题2预测结果进行模型的建立，给定57个分拣中心中并规定有60名正式工，若干个临时工。排班出勤优先使用正式工，临时工针以此来对模型进行动态合理化的调整。最后正式工和临时工的包裹完成量要达到每天基本预期的货量，且在此基础上还得尽可能使出勤天数尽可能少（这里可以参考例子：200名出勤人数（临时工和正式工）出勤30天，最后总人数为6000），再者另一个要求为每天出勤人效尽量均衡，达到稳定数量。

在问题四中，在每个分拣中心出勤人数（正式工和临时工）数量不等情况下，基于问题2预测结果进行模型建立，制定出勤工的未来30天出勤时段表共六个班次，对正式工提出出勤率不能高于0.85的要求，对临时工提出连续出勤不能超过7天的要求。对于其中每个班次，制定出勤表关于多少个正式工出勤于哪个班次，每个班次该雇佣多少临时工，从而完成每天的货量处理。最后要求达成两个目的要求：一是安排的出勤人数尽可能少，二是均衡化每天的实际小时人效和正式工的出勤率.

二、问题分析

2.1 问题一的分析

在问题一中，问题旨在完成两个要求：先是未来30天的时间维度的预测，再是基于此的每个小时预测。对于这种时间序列引起不规律的数值变化，考虑可以采用传统的时间序列模型，完成第一个未来30天的时间预测，然后基于每日货量的预测下完成的每个小时的预测。 ARIMA模型的建立过程包括以下步骤[1]：首先，获取原始的货量数据。紧接着，进行数据预处理，包括空缺值填充和异常值检查。利用python和matlab绘制数据图表，判断数据是否平稳。若数据不平稳，进行d阶差分运算以获取d值。针对平稳的样本时间序列数据，再进行白噪声序列检验，用AIC法则求p，q最优参数，分别运用自相关系数ACF和偏自相关系数PACF，判断自相关系数是否全在置信区间上。最后进行模型预测准备。最后，运用关键参数p、d、q建立ARIMA模型。

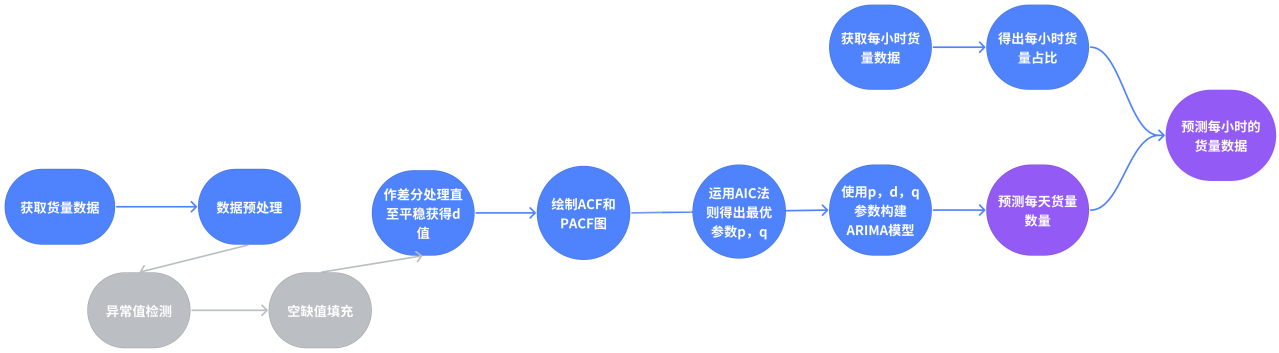


图 1: 问题一的流程

2.2 问题二的分析

在问题二中，基于过去90天的每个分拣中心之间的平均运输量和57个分拣中心未来30天的运输线路之间变化，对其未来天数及其小时的平均运输量影响。线路的变化引起运输量的变化，这里考虑图与平均运输量的模型，将运输线路的变化制定成参数，参数的调整引起整个构建网络物流模型，参数和结构不断更新。最终形成稳定的网络物流模型的预测模型。

- ● 读取数据集，引入图论和网络分析方法
- ● 调整先后分析的预测
- ● 评估节点强度和路径分析
- ● 修正预测，得出每日货量
- ● 根据问题一得出的每小时权重得出每小时的货量

2.3 问题三的分析

在问题三中，这个是一个基于问题2预测结果的人员排班问题。由文中提取出关于三个重要概念：决策变量、约束条件、目标函数。其中决策变量为正式工与临时工的效率。初步考虑约束条件为以下三个：给定57个分拣中心中有60名正式工，若干个临时工；排班出勤优先使用正式工，临时工针对模型进行动态合理化的调整；最后正式工和临时工的完成量要达到每天基本预期的货量；目标函数：要求为每天出勤人效尽量均衡，出勤人数尽可能使出勤天数尽可能少。仅需求解线性不等式即完成优化人员分配的问题[2]。

- ● 选择使用线性规划求解

- ● 确定决策变量
- ● 确定目标函数和约束条件
- ● 根据目标函数和约束条件构造线性规划方程组
- ● 根据方程组求解得出最优的人员分配的方案

2.4 问题四的分析

在问题四中，在问题三中更改了约束条件，决策变量与目标函数。在每个分拣中心出勤人数（正式工和临时工）数量不等情况下，基于问题2预测结果进行模型建立模型。约束条件增加了制定出勤工的未来30天出勤时段表共六个班次，对正式工提出出勤率不能高于百分之85的要求，对正式工提出连续出勤不能超过7天的要求，同时增加了临时工的人数。其中对于每个班次，制定出勤表关于多少个正式工出勤于哪个班次，每个班次该雇佣多少临时工，从而完成每天的货量处理。最后制定两个目标函数：一是安排的出勤人数尽可能少，二是均衡化每天的实际小时人效和正式工的出勤率

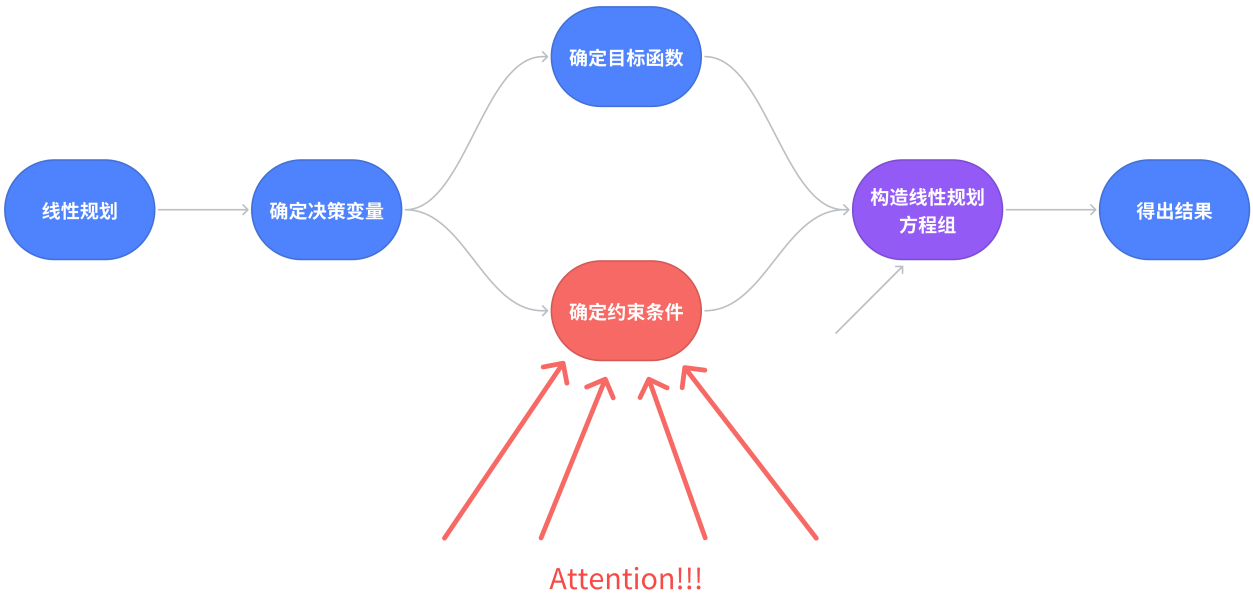


图 2: 问题四的流程

三、模型假设

假设题目附件给的数据真实可靠。
假设所有员工都会按照预期正常工作，不会因为意外、疾病或其他个人原因导致无法工作。

假设分拣中心的运营环境在预测期间保持稳定，没有外部事件影响到货量或运营效率。
假设各个决策变量之间相互独立，没有相互制约或依赖关系。
假设正式工和临时工的工作效率在整个预测期间保持恒定。

四、符号说明

符号	说明	单位
ϵ_t	误差或白噪声	-
$Cov(X,Y)$	随机变量X和Y的协方差	-
$Var(X,Y)$	随机变量X的方差	-
C_i	每天的基本语气货量	件
P	正式工每小时的货量处理能力	件/小时
T	临时工每小时的货量处理能力	件/每小时
$iloc$	预测货量	件

五、模型的建立与求解

5.1 问题一：

5.1.1 问题一中对未来30天每天的货量预测：

数据的预处理： 对数据进行预处理，对于每个分拣中心按照日期进行排序。再使用箱线图进行异常值的查找。箱线图(Box-plot)是用来反映一组或多组连续型定量数据分布的中心位置和散布范围。

中位数（Q_2 / 50th Percentile）:数据集的中间值；代表了样本数据的平均水平。

第一个四分位数（Q_1 / 25th Percentile）:最小数和数据集的中位数之间的中间数。

第三四分位数（Q_3 / 75th Percentile):数据集的中位数和最大值之间的中间值。

四分位间距（IQR）:第25至第75个百分点的距离；也即盒子的大小/箱体长度，箱子包含了50的数据，箱子的高度在一定程度上反映了数据的波动程度，箱体越扁说明数据越集中。

四分位差

$$IQR = Q_3 - Q_1$$

上边缘：

$$Q_3 + 1.5IQR = max(xi : xi \leq Q_3 + 1.5IQR)$$

下边缘：

$$Q_1 - 1.5IQR = min(xi : xi \geq Q_1 - 1.5IQR)$$

如果数据有存在离群点即异常值，他们超出上下边缘，此时将离群点以“圆点”形式进行展示，超过上下边缘的点被称为异常值点。

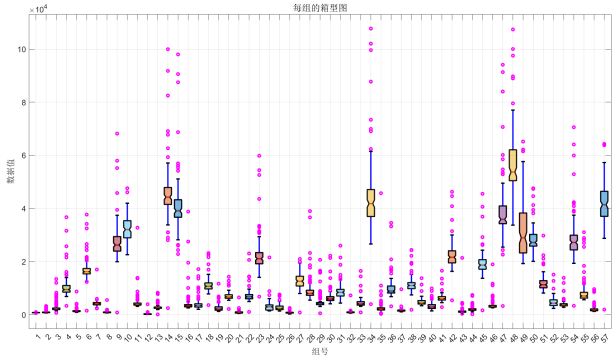


图 3: 57组数据的箱型图

对于偏离过多的数据，可能会影响预测的结果，将异常值数据删去，使用拉格朗日插值法将缺失的值填补进来。对某个多项式函数，已知有给定的$k+1$个取值点：

$$(x_0, y_0), (x_1, y_1), \cdots, (x_k, y_k)$$

其中，x_j对应着自变量的位置，而y_j对应着函数在这个位置的取值。假设任意两个不同的x_j都互不相同，那么应用拉格朗日插值公式所得到的拉格朗日插值多项式[3]为：

$$L(x) := \sum_{j=0}^{k} y_j l_j(x)$$

其中，每个$l_j(x)$为拉格朗日基本多项式,表达式为

$$l_j(x) := \prod_{i=0, i\neq j}^{k} \frac{x - x_i}{x_j - x_i} = \frac{x - x_0}{x_j - x_0} \cdots \frac{x - x_{j-1}}{x_j - x_{j-1}} \frac{x - x_{j+1}}{x_j - x_{j+1}} \cdots \frac{x - x_k}{x_j - x_k}$$

拉格朗日基本多项式$l_j(x)$的特点是在x_j上取值为1，在其他点x_i,$i \neq j$上取值为0。将所有的缺失值进行插值后，数据预处理完成。

模型的建立与求解： 使用ARIMA模型预测接下来的30天的运货量，此模型是一个非常灵活的模型，对于时间序列好多特征都能够进行描述。ARIMA模型分成三个部分：AR，MR，I三个模块的组合。

MA（移动平均模型）：MA（q）假设一个时间序列由误差项以及这些噪音滞后阶根据不同的权重组合起来，计算公式如下：

$$Y_t = \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}$$

MA移动平均模型关注的是自回归模型中的误差项的累加

AR（自回归模型）：一般的p阶自回归模型：

$$X_t = \alpha_1 X_{t-1} + \alpha_2 X_{t-2} + \cdots + \alpha_p X_{t-p} + u_t$$

AR(p):假设今天的运货量依赖于前一天的运货量或者依赖于前p天的运货量，前p天的影响到了今天的运货量有以下公式：

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + \epsilon_t$$

自回归模型需要确定阶数p,表示用几期历史值来预测当前值，使用循环迭代来确定p的具体值

ARMA(自回归移动平移模型):ARMA同时受AR和MA影响，我们将自回归模型AR和移动平移模型MA综合成ARMA，公式如下：

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \phi_3 Y_{t-3} + \cdots + \alpha_t + \theta_1 \alpha_{t-1} + \theta_2 \alpha_{t-2} + \theta_q \alpha_{t-q}$$

ARMA是非常有效的时间序列检测工具，是一个线性模型，所以我们需要检测数据是否符合线性趋势。

ARIMA（自回归移动平均模型ARIMA）：I既是差分项，如果数据不符合线性趋势，将进行差分直至符合要求，将不平稳的序列变平稳，差分形式：

$$(1 - \Phi(L))(1 - L)^d Y_t = (1 + \Theta(L))\epsilon_t$$

例如SC10的过去货量折线图：

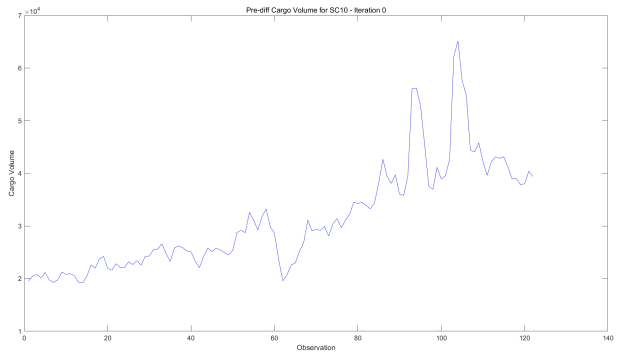


图 4: SC10原数据

可以观察出数据是不平稳的，为有一个相对客观评判数据是否平稳的指标，我们使用ACF和PACF来进行评判过去的数据是否达到标准。

ACF（样本自相关函数）：展现t时刻和t-k时刻时间序列的关联性，描述的是时间序列观测值与其过去的观测值之间的线性相关性。

$$\rho = \frac{Cov(y_t, y_{t-k})}{\sqrt{Var(y_t)Var(y_t - k)}} = \frac{\gamma_k}{\gamma_k}$$

$$\rho = 1, \rho_k = \rho_{-k}, -1 \leqslant \rho \leqslant 1$$

PACF(样本偏自相关函数)：在给定$y_{t-1}, y_{t-2}, \cdots, y_{t-k+1}$的条件下， y_t与滞后 k 期时间序列之间的条件相关。

$$E[(Y_t - \mu_t)(Y_{t-k} - \mu_{t-k})|Y_{t-1}, Y_{t-2}, \cdots, Y_{t-k+1}]$$

$$y_t = \alpha_0 + \alpha_1 y_{t-1} + \alpha_2 y_{t-2} + \alpha_3 y_{t-3}$$

α_1 α_2 α_3分别是1阶滞后，2阶滞后，3阶滞后下的PACF。PACF描述的是在给定中间观测值的条件下，时间序列观测值预期过去的观测值之间的线性相关性。例如SC10的ACF和PACF：

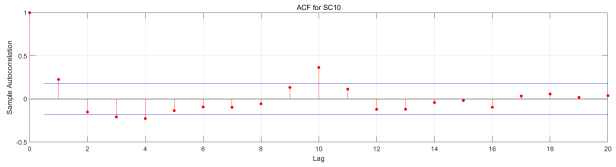


图 5: SC10的ACF

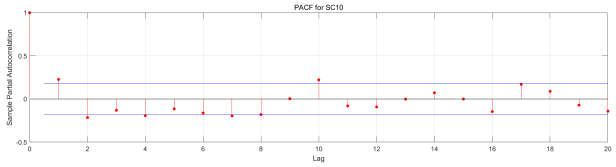


图 6: SC10的PACF

将参数设置为0.05，当满足参数条件时即可认为数据平稳，如果数据不平稳，将反复进行差分直至数据通过平稳性检验。将57个站点的数据均进行上述处理，例如SC10处理后的数据图像如下：

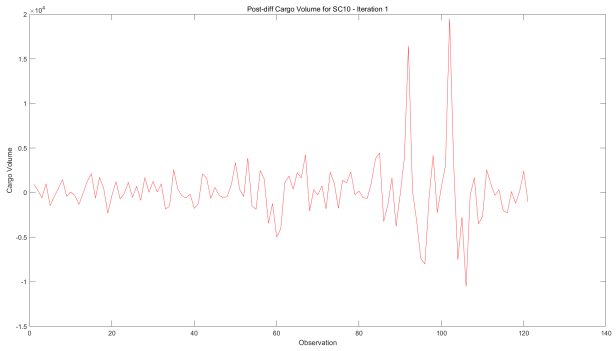


图 7: SC10差分后的数据

ARIMA模型：在ARMA的基础上做了差分，上述求出差分后的数据，将数据转化为平稳的序列，在AMRA的基础上进行回归预测：例如SC10的后30天的预测结果：

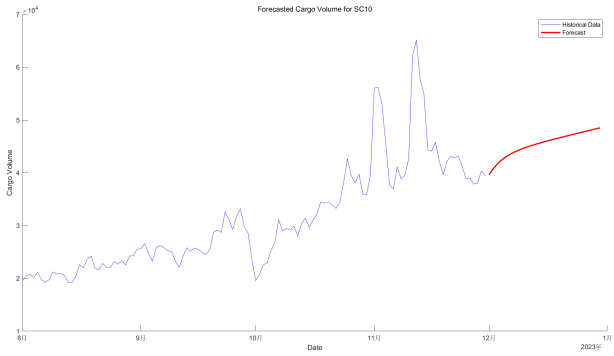


图 8: SC10的预测结果

将给出的57个站点分别进行上述步骤预测，即可得出这57站点的后30天运货量的预测值。

5.1.2 问题一中对未来30天每小时的货量预测：

对数据进行处理，对于每个分拣中心，按照日期小时进行排序.对每个分拣中心内部按日期排序。数据中存在缺失的值，找到对应缺失数据对应的值将其赋值为0 。以运货量为指标，求出每小时运货量的权重，根据预测的每天的结果×权重，即可求出每天每小时的运货量。由于横坐标数量过多数据可视化的趋势与每天的预测趋势图相同不利于观察具体每小时的变化，即举例说明，例如SC10第一天的每小时预测如图：

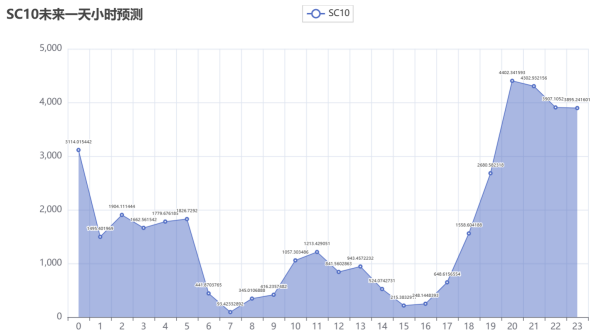


图 9: SC10第一天每小时的预测结果

SC10第二天的每小时预测如图：



图 10: SC10的第二天每小时的预测结果

57组数据同上述操作均可预测出未来30天每小时的预测数据。

5.2 问题二：

5.2.1 问题二中对未来30天每天的货量预测：

图论(Graph)是由瑞士数学家欧拉于 1736 年发表的论文提出的概念。它是由顶点的有穷非空集合和顶点之间边的集合组成。其结构记为G(V,E)。在图的结构中，结点的集合与边的集合之间按照一定对应关系相连接组成无向图，若给无向图的边附上权重即为有向图。若干个附上权重的有向图也称为网络。附上的权重现实意义可以指费用、时间、价钱、利润等。而本文中，我们将货量作为网络中相应的权重基于[4]：

$$k_i = \sum_{j=1}^{N} a_{ij} = \sum_{j=1}^{N} a_{ji},$$

k_i: 表示节点i的度，即与节点i相连的边的数量之和。 N: 表示图中节点的总数。 a_{ij}: 表示节点i和节点j之间的连接权重。 $< k >$: 表示图中所有节点的平均度，即所有节点度的总和除以节点总数。

通过上述构建图形网络物流模型，旨在利用历史数据进行分析实现对未来货量的准确预测。我在本文中引入节点强度概念，它是以分拣中心为节点和分拣中心之间的权重为边为整体的总和。另外在这个图形网络物流模型中的边的权重表示货量。引进出度节点强度概念，考量从该节点出发到其他节点的边的权重之和，构建一个包含N个节点的加权网络G：

$$s_i^{out} = \sum_{j=1}^{N} w_{ij}, s_i^{in} = \sum_{j=1}^{N} w_{ij}$$

s_i^{out}: 表示节点i的出度节点强度，即从节点i出发到其他节点的边的权重之和。 s_i^{in}: 表示节点i的入度节点强度，即到达节点i的边的权重之和。 w_{ij}: 表示节点i到节点j之间的边的权重。

通过读取附件3文件，我们可以获取到物流网络中的边和相应的货量信息和读取附件4文件，我们可以获取到未来可能新增的边。接着使用$python$函数创建有向图G，其中边的起点为始发分拣中心，终点为到达分拣中心，边的权重为货量。本文基于$Networkx$提供了可视化图的基本功能使用$Networkx$库构建物流网络模型。

为了适应物流网络的变化，我们计算每个节点对其邻节点的相对货物流动量的比率来调整这些预测后的货量。比如，如果SC10节点的总强度是通过两条边的货物量计算得出，并且预测SC10将处理标准单位的货量，则其流向每个邻居节点的货量将根据它们各自的边权重（在总强度中的比例）来分配。简述来说，我们制定一个原则：若该带点分拣中心在过去历史中获得较大历史货量，那么在未来以后将会获得相应邻居节点的较大的权重。我们需要不断更新网络图的结构包括节点强度和优化整个网络图的结构。计算每个邻居二点分配比例：

对于节点v和他的邻居节点u，分配给u的货量比例为$w(v,u)$与v的总出度强度为:

$$proportion_{v\to u} = \frac{w(v,u)}{\sum_{(v,x)\in E} w(v,x)}$$

货量的具体分配：

对于每个节点v和其邻居u，计算实际分配给u的货量：

$$allocatedload(v \to u) = proportion(v \to u) \times futuretotalload$$

分配给u的货量为：

$$allocatedload(v \to u) = (\frac{w(v,u)}{\sum_{u,x} w(v,x)}) \times futuretotalload$$

在每一对始发分拣中心和到达分拣中心，我们分两种情况，一种是原先不存在这份货量（权重），另一种是，原先已存在权重我们检查它们之间是否已经存在边。如果货量（权重）存在，则计算与这两个分拣中心相关的所有货量（权重），并更新这条边的货量（权重）。如果货量（权重）不存在，我们便计算平均货量，并添加这条新货量（权重）到这个图形网络物流模型。

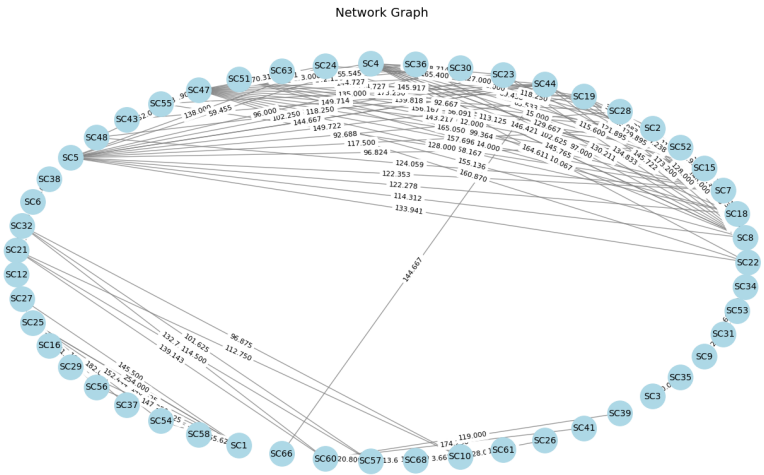


图 11: 构建网络图

我们使用pandas库读取CSV文件，该文件包含了每日的货量预测。为了确保数据的一致性，我们将日期列转换为datetime格式。接着，我们创建一个新的数据框，用于存储更新后的货量。通过遍历图中的每条边，我们根据每日的预测数据调整货量，并更新数据框。如表为SC10的预测结果：

表 1: SC10的预测结果

日期	货量	日期	货量
2023/12/1	40056.70283	2023/12/16	39878.52653
2023/12/2	40062.80295	2023/12/17	39877.27236
2023/12/3	40007.25623	2023/12/18	39878.3776
2023/12/4	39930.84464	2023/12/19	39880.61881
2023/12/5	39866.41601	2023/12/20	39882.84198
2023/12/6	39830.91084	2023/12/21	39884.31875
2023/12/7	39825.54737	2023/12/22	39884.83238
2023/12/8	39841.13672	2023/12/23	39884.56335
2023/12/9	39864.90536	2023/12/24	39883.88524
2023/12/10	39886.02135	2023/12/25	39883.17099
2023/12/11	39898.46899	2023/12/26	39882.66985
2023/12/12	39901.28592	2023/12/27	39882.46998
2023/12/13	39897.04341	2023/12/28	39882.52789
2023/12/14	39889.7113	2023/12/29	39882.73069
2023/12/15	39882.83702	2023/12/30	39882.95864

5.2.2 问题二中对未来30天每小时的货量预测：

我们需要将每日预测货量细化到每小时。通过读取CSV文件，即问题一中的小时预测中得出的每小时比例，我们获取到每个分拣中心的小时货量分布比例。定义了一个函数，该函数将每日货量预测与小时分布比例合并，计算出每小时的预测货量。最后我们将结果保存到新的CSV文件中。

表 2: SC10的预测结果

时间	货量	时间	货量
0	3156.480268	12	853.0363729
1	1515.794284	13	956.3228456
2	1930.077198	14	531.2209052
3	1685.233358	15	218.3204043
4	1803.945056	16	251.5287105
5	1851.639718	17	657.4606182
6	447.8960206	18	1579.858371
7	94.69731277	19	2717.136556
8	349.7154884	20	4462.374908
9	421.911821	21	4361.609856
10	1071.721593	22	3960.385212
11	1229.976192	23	3948.359757

5.3 问题三：

线性规划[5]是数学规划的一个重要的分支。变量称为决策变量，规划的目标称为目标函数，限制条件称为约束条件。线性规划模型的一般形式：

$$\max(\text{or } \min)\ z = c_1x_1 + c_2x_2 + \cdots + c_nx_nz$$

$$s.t\begin{cases} a_{11}x_1 + a_{12}x_1 + \cdots + a_{1n}x_n \le (or =, \ge)b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \le (or =, \ge)b_2, \\ \cdots \\ a_{m1}x_1 + a_{m2}x_1 + \cdots + a_{mn}x_n \le (or =, \ge)b_m, \\ x_1, x_2, \cdots, x_n \ge 0 \end{cases}$$

根据时间段，将时间分为9个区间，每个区间有重叠的部分：

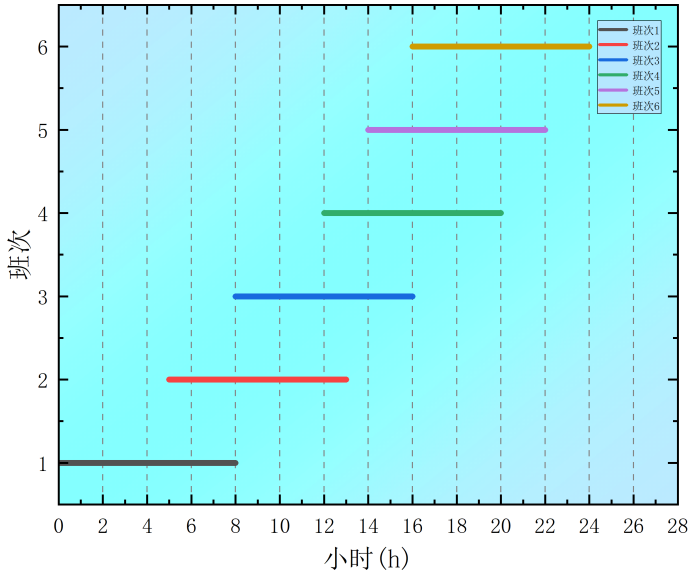


图 12: 时间段

决策变量：

设x_{ij}:第i天第j个班次的正式工人数且为整数。

设y_{ij}:第i天第 j个班次的临时工人数且为整数。

设z_{ij}:第i天第 j个班次的的效率偏差且为连续变量。

画出每天的小时货量，以SC10为例：

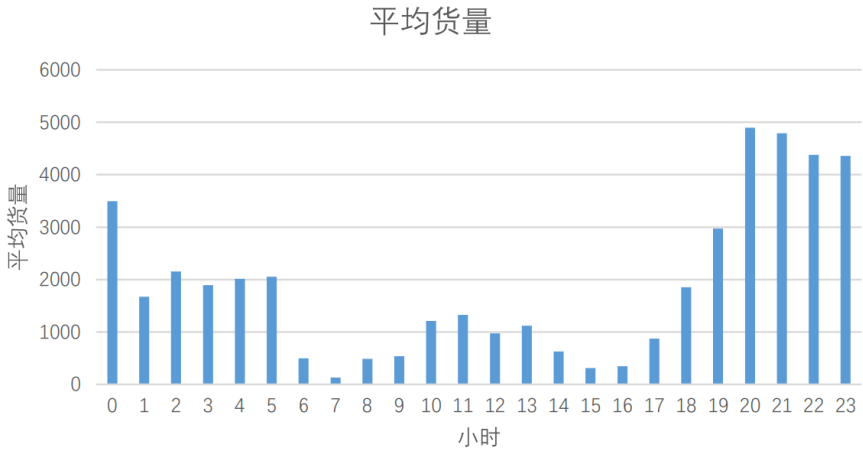


图 13: SC10

可以看出，时间段在0至5小时和18至23小时货量相对多，以次方便得出约束条件。

目标函数：最小化总人天数和效率偏差:

$$Z = \sum i \sum j(x_{ij} + y_{ij} + z_{ij})$$

约束条件: 根据题目可知正式工每天最多出勤一个班次，即：

$$\sum j x_{ij} = 1 \qquad \forall i$$

人员的非负性，人数不可能小于零，即：

$$x_{ij} \geq 0 \quad y_{ij} \geq 0 \quad \forall i, j$$

正式工的使用人数不可以超过60人，即：

$$x_{ij} \leq \quad \forall i, j$$

每班次的总分拣量不可以少于预测货量，即：

$$P \cdot x_{ij} + T \cdot y_{ij} \geq 6 \cdot C_i quad \forall i, j$$

确保效率保持在一个合理的范围,员工的最高平均每小时人效（效率）定义为：$target_efficiency \cdot (x_{ij} + y_{ij} + 0.1)$:

$$target_efficiency = P \cdot \frac{P}{P+T} + \frac{T}{P+T} \cdot T$$

处理效率的偏差为：

$$|P \cdot x_{ij} + T \cdot y_{ij} - target_efficiency \cdot (x_{ij} + y_{ij} + 0.1)| \leq z_{ij} \quad \forall i, j$$

根据目标函数约束添加和决策变量即可得出线性规划式：

$$\min Z = \sum i \sum j(x_{ij} + y_{ij} + z_{ij})$$

$$s.t \begin{cases} \sum j x_{ij} = 1 \quad \forall i \\ x_{ij} \geq 0 \quad y_{ij} \geq 0 \quad \forall i, j \\ x_{ij} \leq \quad \forall i, j \\ P \cdot x_{ij} + T \cdot y_{ij} \geq 6 \cdot C_i \quad \forall i, j \\ |P \cdot x_{ij} + T \cdot y_{ij} - target_efficiency \cdot (x_{ij} + y_{ij} + 0.1)| \leq z_{ij} \quad \forall i, j \end{cases}$$

以此得出结果[6]，以SC10第一天为例：

表 3: SC10的预测结果

时间段	正式工人数	临时工人数
00:00-08:00	52	93
05:00-13:00	2	11
08:00-16:00	0	48
12:00-20:00	0	0
14:00-22:00	0	25
16:00-24:00	6	191

5.4 问题四：

同样是使用线性规划来进行求解，在第三问的基础上增加或变化了约束条件。

1：每名正式员工出勤率不能高于百分之85，且出勤天数不能连续超过7天，同时增加了正式工的人数，此问题和问题三类似，但是更加宏观的去考虑正式工的出勤问题，这里可以使正式工工作5天休息一天（30天刚好休息5天），出勤率刚好到达百分之83.3，同时也能满足正式工不能连续出勤7天的要求，然后按批次去对其进行排班即可。

2.SC60的正式员工有200名。

决策变量：

1.正式工的半侧变量：

$$x_i, \quad i \in 1,2,3,4,5$$

2.临时工的班次变量：

$$y_i, \quad i \in 1,2,3,4,5$$

3.正式工的休息日变量：

$$rest_i, \quad \in 1,2,3,4,5$$

目标函数：

$$min \quad objective = x_1 + x_2 + x_3 + x_4 + x_5 + \sum_{i=1}^{5} rest_i$$

约束条件： 1.正式工的休息日约束：

$$\sum rest_i \geq 1$$

2.总工作量不能低于预测货量约束：

$$25x_i + 20y_i \geq iloc$$

3.正式工的出勤人数不能超过200人约束：

$$x_i + x_2 + x_3 + x_4 + x_5 \leq 200$$

4.将正式员工的休息日变量添加到目标函数中的约束：

$$objective+ = rest_1 + rest_2 + rest_3 + rest_4 + rest_5$$

所以线性规划表达式为：

$$\min \quad objective = x_1 + x_2 + x_3 + x_4 + x_5 + \sum_{i=1}^{5} rest_i$$

$$s.t \begin{cases} \sum rest_i \geq 1 \\ 25x_i + 20y_i \geq iloc \\ x_i + x_2 + x_3 + x_4 + x_5 \leq 200 \\ objective+ = rest_1 + rest_2 + rest_3 + rest_4 + rest_5 \end{cases}$$

求解得出SC60的员工安排结果：

表 4: 举例SC60第一天天的员工安排

时间段	正式工人数	临时工人数
00:00-08:00	173	9
05:00-13:00	0	0
08:00-16:00	3	83
12:00-20:00	0	19
14:00-22:00	0	0
16:00-24:00	24	198

表 5: 举例SC60第二天的员工安排

时间段	正式工人数	临时工人数
00:00-08:00	177	2
05:00-13:00	0	0
08:00-16:00	0	105
12:00-20:00	0	0
14:00-22:00	0	0
16:00-24:00	23	197

表 6: 举例SC60第三天的员工安排

时间段	正式工人数	临时工人数
00:00-08:00	176	2
05:00-13:00	0	0
08:00-16:00	3	104
12:00-20:00	0	0
14:00-22:00	0	0
16:00-24:00	24	194

六、模型的评价与推广

6.1 模型的优点：

在问题1和问题2中，使用ARIMA模型进行预测可以捕捉影响货量的关键因素，通过设置合理的参数来解决实际问题。这种方法可以有效地分析和预测时间序列数据，为决策提供有力支持。

问题3和问题4主要通过建立数学模型，将复杂的问题简化为寻找决策函数和目标函数等相对简单的问题来解决。通过线性规划的方法，可以有效地优化资源分配、制定最佳决策方案等，为问题的求解提供了一种有效且可行的途径。这种方法能够帮助在面对复杂情况时快速找到最优解，提高决策效率和准确性。

本文所使用的算法和模型具有高效性和稳定性，能够在处理数据时保持较高的效率。这些算法和模型经过精心设计和优化，能够快速而准确地处理输入数据，提供可靠的结果。它们在解决实际问题时表现出色，在实际应用中具有一定的效率和稳定性。

6.2 模型的缺点：

· 对于预测模型，货量的数量可能还会与季节相关，例如第四季度明显高于第二季度，数据呈现非相关性，ARIMA模型适合处理线性模型，本文将其差分后得出相对线性数据再进行ARIMA处理，可能存在一定的误差。

6.3 模型的推广：

在预测方面，可以将ARIMA模型替换成LSTM模型，从而更好的解决非线性预测问题。通过使用LSTM模型，可以更好地捕捉数据中的复杂关联和动态模式，从而提高预测的准确性和泛化能力。LSTM模型能够适应数据中的非线性变化和趋势，同时能够处理长期依赖关系，使其在预测具有复杂结构和非线性特征的时间序列数据时表现更为优越。

参考文献

[1] 李玲玲,辛浩.时序模型ARIMA在数据分析中的应用[J].福建电脑,2024,40(04):25-29.DOI:10.16707/j.cnki.fjpc.2024.04.006.

[2] Cottle R W. George B. Dantzig: Operations research icon. Oper Res, 2005, 53: 892–898

[3] 刘九州,王昆.功能重置成本法与拉格朗日插值[J].检疫检验科学,2001.5.

[4] Douglas B.West[美].图论导引[M].北京.机械工业出版社，2006

[5] 孙文瑜, 徐成贤, 朱德通.最优化方法(第二版)[M]. 北京:高等教育出版社, 2010.

[6] 线性规划在多元线性回归中的解法及应用。张爱兰，唐虹。昆明理工大学学报（自然科学版）39卷第1期，2014年2月

附录

问题一中每天运货量预测：matlab源代码

```
1
2  % 加载数据，没有变量名，从第一行开始数据Excel
3  data = readtable('fujian1.1.xlsx', 'ReadVariableNames', false);
4  % 假设第一列是分拣中心的名称
5  sortingCenters = unique(data{:, 1});
6  % 预先定义时间格式
7  dateFormat = 'yyyy-MM-dd'; % 根据实际情况调整日期格式
8  % 循环遍历所有的分拣中心
9  for i = 1:length(sortingCenters)
10 centerName = sortingCenters{i};
11 % 提取当前分拣中心的数据
12 centerData = data(strcmp(data{:, 1}, centerName), :);
13 Y = centerData{:, 3}; % 货量数据
14 dates = centerData{:, 2}; % 日期数据
15 % 将日期转换为日期格式MATLAB
16 dates = datetime(dates, 'InputFormat', dateFormat);
17 % 绘制原始数据图像
18 figure;
19 plot(dates, Y, 'b');
20 title(['Original Cargo Volume for ' centerName]);
21 xlabel('Date');
22 ylabel('Cargo Volume');
23 % 检查时间序列的平稳性并进行差分处理
24 Y_diff = Y;
25 d = 0;
26 while ~adftest(Y_diff, 'Alpha', 0.05)
27 % 在进行差分之前绘制当前差分序列的图像
28 figure;
29 plot(Y_diff, 'b');
30 title(['Pre-diff Cargo Volume for ' centerName ' - Iteration '
31 num2str(d)]);
32 xlabel('Observation');
33 ylabel('Cargo Volume');
34 Y_diff = diff(Y_diff);
35 d = d + 1;
36 % 绘制差分后的数据图像
```

```
figure;
plot(Y_diff, 'r');
title(['Post-diff Cargo Volume for ' centerName ' - Iteration '
num2str(d)]);
xlabel('Observation');
ylabel('Cargo Volume');
end
% 绘制和ACFPACF
figure;
subplot(211);
autocorr(Y_diff);
title(['ACF for ' centerName]);
subplot(212);
parcorr(Y_diff);
title(['PACF for ' centerName]);
% 自动选择和pq
bestAIC = Inf;
bestP = 0;
bestQ = 0;
for p = 0:5  % 测试的值从到p05
for q = 0:5  % 测试的值从到q05
try
model = arima('Constant', 0, 'D', d, 'ARLags', 1:p,
'MALags', 1:q);
[fit,~,logL] = estimate(model, Y_diff, 'Display', 'off');
numParams = p + q + 1;
aic = aicbic(logL, numParams);
if aic < bestAIC
bestAIC = aic;
bestP = p;
bestQ = q;
end
catch
% 捕获并忽略由不适合的模型引起的错误
end
end
end
% 使用最佳的和拟合模型pqARIMA
model = arima('Constant', 0, 'D', d, 'ARLags', 1:bestP,
```

```
'MALags', 1:bestQ);
[EstModel, EstParamCov] = estimate(model, Y_diff,
'Display', 'off');
% 使用拟合的模型进行预测
numDays = 30; % 预测未来天的货量30
[YPred, YMSE] = forecast(EstModel, numDays, 'Y0', Y_diff);
% 将预测的差分数据转换回原始的数据规模
Y_forecast = [Y(end); YPred];
for j = 2:length(Y_forecast)
Y_forecast(j) = Y_forecast(j) + Y_forecast(j - 1);
end
Y_forecast(1) = []; % 删除第一个元素，因为它是历史数据的最后一个点
% 保存预测结果
disp(['Predicted Cargo Volumes for ' centerName ':']);
disp(Y_forecast);
% 可视化预测结果
figure;
hold on;
plot(dates, Y, 'b');
plot(dates(end) + (1:numDays), Y_forecast, 'r', 'LineWidth', 2);
hold off;
title(['Forecasted Cargo Volume for ' centerName]);
xlabel('Date');
ylabel('Cargo Volume');
legend('Historical Data', 'Forecast');
end
filename = 'fujian1.csv';
data = readmatrix(filename, 'Range', 'C:C');
groupSize = 122;
numGroups = floor(length(data) / groupSize);
% 存储异常值
outliers = [];
% 准备绘制箱型图的数据
boxplotData = zeros(groupSize, numGroups); % 初始化为零的矩阵
% 遍历每组数据
for i = 1:numGroups
% 获取当前组的数据
startIndex = (i - 1) * groupSize + 1;
endIndex = i * groupSize;
```

```
groupData = data(startIndex:endIndex);
% 使用箱线图计算异常值
lowerInnerFence = quantile(groupData, 0.25)
- 1.5 * iqr(groupData);
upperInnerFence = quantile(groupData, 0.75)
+ 1.5 * iqr(groupData);
groupOutliers = groupData(groupData < lowerInnerFence
| groupData > upperInnerFence);
% 存储异常值和所在组索引
if ~isempty(groupOutliers)
outliers = [outliers; repmat(i, length(groupOutliers), 1)
,groupOutliers];
end
% 使用拉格朗日插值填充异常值
x = 1:groupSize;
y = groupData;
% 找到异常值的索引
outlierIndices = find(groupData < lowerInnerFence
| groupData > upperInnerFence);
% 逐个替换异常值
for j = 1:length(outlierIndices)
outlierIndex = outlierIndices(j);
% 使用拉格朗日插值进行插值填充
interpolatedValue = lagrange_interp(x, y, outlierIndex);
% 替换异常值
groupData(outlierIndex) = interpolatedValue;
end
% 更新原数据集
data(startIndex:endIndex) = groupData;
% 存储箱型图数据
boxplotData(:, i) = groupData;
end
% 绘制美化后的箱型图
figure; % 创建一个新的图形窗口
boxplotHandle = boxplot(boxplotData, 'Notch', 'on',
'Labels', arrayfun(@num2str, 1:numGroups, 'UniformOutput', false),
'Whisker', 1.5, 'Symbol', 'ro', 'OutlierSize', 4, 'Widths', 0.7);
set(boxplotHandle, {'linew'}, {2}); % 设置线宽
set(gca, 'FontSize', 12); % 设置坐标轴字体大小
```

```
154 title('每组的箱型图', 'FontSize', 14); % 设置标题和字体大小
155 xlabel('组号', 'FontSize', 13); % 设置轴标签和字体大小X
156 ylabel('数据值', 'FontSize', 13); % 设置轴标签和字体大小Y
157 grid on; % 添加网格线
158 % 改变箱体颜色
159 set(findobj(gca, 'Type', 'line', 'Tag', 'Median'),
160  'Color', 'r'); % 中位数线颜色
161 set(findobj(gca, 'Type', 'line', 'Tag', 'Box'),
162  'Color', 'k'); % 箱体边框颜色
163 set(findobj(gca, 'Type', 'line', 'Tag', 'Upper␣Whisker'),
164  'Color', 'b', 'LineStyle', '–'); % 上须颜色和样式
165 set(findobj(gca, 'Type', 'line', 'Tag', 'Lower␣Whisker'),
166  'Color', 'b', 'LineStyle', '–'); % 下须颜色和样式
167 % 改变异常值的颜色和样式
168 hOutliers = findobj(gca, 'Tag', 'Outliers');
169 for iOutlier = 1:length(hOutliers)
170 set(hOutliers(iOutlier), 'MarkerEdgeColor', 'm'); % 异常值颜色
171 end
172 colors = lines(numGroups); % 生成颜色
173 h = findobj(gca, 'Tag', 'Box');
174 for j=1:length(h)
175 patch(get(h(j), 'XData'), get(h(j), 'YData'),
176  colors(j,:), 'FaceAlpha', .5);
177 end
178 function yi = lagrange_interp(x, y, xi)
179 n = length(x);
180 m = length(xi);
181 yi = zeros(m, 1);
182 for j = 1:m
183 numerator = ones(n, 1);
184 denominator = ones(n, 1);
185 for i = 1:n
186 for k = 1:n
187 if i ~= k
188 numerator(i) = numerator(i) * (xi(j) - x(k));
189 denominator(i) = denominator(i) * (x(i) - x(k));
190 end
191 end
192 end
```

```
yi(j) = sum(y .* numerator ./ denominator);
end
end
```

问题一中每天每小时的运货量预测：**python**源代码

```
import pandas as pd
import numpy as np
from statsmodels.tsa.stattools import adfuller, acf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox
def analyze_and_forecast(df, center):
    data = df[df['分拣中心'] == center]
    series = data.set_index('日期')['货量']
    series.index.freq = 'D'
    def check_stationarity(timeseries):
        d = 0
        result = adfuller(timeseries)
        while result[1] > 0.05:
            timeseries = timeseries.diff().dropna()
            result = adfuller(timeseries)
            d += 1
        return timeseries, d

    stationary_series, d = check_stationarity(series)
    if acorr_ljungbox(stationary_series, lags=[3],
    return_df=True)['lb_pvalue'].iloc[0] > 0.05:
        print(f'{center} 序列可能是白噪声')
        return
    pmax = 4
    qmax = 4
    bic = np.inf
    best_p, best_q = 0, 0

    for p in range(pmax + 1):
        for q in range(qmax + 1):
            try:
                model = ARIMA(stationary_series, order=(p, d, q))
                results = model.fit()
```

```
                if results.bic < bic:
                    bic = results.bic
                    best_p, best_q = p, q
            except:
                continue
    best_model = ARIMA(series, order=(best_p, d, best_q))
    fitted_model = best_model.fit()
    forecast = fitted_model.get_forecast(steps=30)
    forecast_df = forecast.summary_frame()
    result_df = pd.DataFrame({
        '分拣中心': [center] * 30,
        '日期': pd.date_range(start=series.index[-1] +
            pd.Timedelta(days=1), periods=30, freq='D'),
        '货量': forecast_df['mean'].values
    })
    return result_df
def forecast_all_centers(df):
    results = []
    centers = df['分拣中心'].unique()
    for center in centers:
        result = analyze_and_forecast(df, center)
        if result is not None:
            results.append(result)
    final_result = pd.concat(results)
    return final_result
final_result = forecast_all_centers(Data_day)
final_result.to_csv('answer1_1.csv',index=False)
Data_day_hour = pd.read_excel('附件2.xlsx')
Data_day_hour.head()
Data_day_hour['日期'] = pd.to_datetime(Data_day_hour['日期'])
Data_day_hour = Data_day_hour.sort_values
(['分拣中心', '日期','小时'])
Data_day_hour.reset_index(drop=True,inplace=True)
def fill_missing_hours(df):
    df['小时'] = df['小时'].astype(int)
    centers = df['分拣中心'].unique()
    dates = df['日期'].unique()
    filled_df = pd.DataFrame()
    for center in centers:
```

```
        for date in dates:
            df_subset = df[(df['分拣中心'] == center)
            & (df['日期'] == date)]
            all_hours = pd.DataFrame({'小时': range(24)})
            all_hours['分拣中心'] = center
            all_hours['日期'] = date
            df_full_hours = all_hours.merge(df_subset,
            on=['分拣中心', '日期', '小时'], how='left')
            df_full_hours['货量'] = df_full_hours['货量'].fillna(0)
            filled_df = pd.concat([filled_df,
            df_full_hours], ignore_index=True)
    return filled_df
Data_day_hour_filled = fill_missing_hours(Data_day_hour)
def calculate_hourly_distribution(df):
    total_daily = df.groupby(['分拣中心', '日期']).
    agg({'货量': 'sum'}).rename(columns={'货量': '日总货量'})
    hourly_data = df.merge(total_daily, on=['分拣中心', '日期'])
    hourly_data['小时比例'] = hourly_data['货量'] / hourly_data['日
总货量']
    avg_hourly_distribution = hourly_data.groupby(['分拣中心', '小
时']).
    agg({'小时比例': 'mean'}).reset_index()
    return avg_hourly_distribution
def distribute_daily_to_hourly(daily_forecast, hourly_distribution):
    forecast_detailed = daily_forecast.merge(hourly_distribution,
    on=['分拣中心'])
    forecast_detailed['预测货量'] = forecast_detailed['货量']
    * forecast_detailed['小时比例']
    return forecast_detailed[['分拣中心', '日期', '小时', '预测货量']]
avg_hourly_distribution = calculate_hourly_distribution
(Data_day_hour_filled)
avg_hourly_distribution.to_csv('avg_distribution.csv',index=False)
final_result_hourly = distribute_daily_to_hourly
(final_result,avg_hourly_distribution)
final_result_hourly.to_csv('answer1_2.csv',index=False)
```

问题二中每天运货量预测：**python源代码**

```
import numpy as np
import pandas as pd
import networkx as nx
```

```
edges_df = pd.read_csv('附件3.csv')
edges_df.head()
G = nx.from_pandas_edgelist(
edges_df,
source='始发分拣中心',
target='到达分拣中心',
edge_attr='货量',
create_using=nx.DiGraph()
)
future_edges_df = pd.read_csv('附件4.csv')
# 更新网络图
for _, row in future_edges_df.iterrows():
    src, dst = row['始发分拣中心'], row['到达分拣中心']
    # 检查边是否存在，存在则更新，不存在则添加
    if G.has_edge(src, dst):
        # 使用与该目的地或来源相关的所有边的平均货量
        specific_avg = edges_df[
            (edges_df['始发分拣中心'] == src) | (edges_df['到达分拣
中心'] == dst)
        ]['货量'].mean()
        G[src][dst]['货量'] = specific_avg
    else:
        # 对于不存在的边，也计算与目的地或来源相关的边的平均货量
        specific_avg = edges_df[
            (edges_df['始发分拣中心'] == src) | (edges_df['到达␣分拣
中心'] == dst)
        ]['货量'].mean()
        G.add_edge(src, dst, 货量=specific_avg if not
        pd.isna(specific_avg) else 0)
# 打印图中所有边的信息
for edge in G.edges(data=True):
    print(edge)
daily_prediction = pd.read_csv('answer1_1.csv')
```

问题二中每天每小时的运货量预测：python源代码

```
import pandas as pd
import networkx as nx
# 转换日期格式，确保一致性
daily_prediction['日期'] = pd.to_datetime
(daily_prediction['日期'])
```

```
# 创建空用于存储更新后的货量DataFrame
updated_loads = daily_prediction.copy()
# 将转换为每天一个字典，便于操作DataFrame
daily_load_dict = daily_prediction.groupby('日期').apply
(lambda x: dict(zip(x['分拣中心'], x['货量']))).to_dict()
# 遍历每天，调整货量
for date, loads in daily_load_dict.items():
    # 遍历图中每条边，调整货量
    for (src, dst, data) in G.edges(data=True):
        # 确保源和目的节点都在当天的预测数据中
        if src in loads and dst in loads:
            # 计算要转移的货量
            transfer_amount = data['货量']
        # 从源分拣中心减去货量
        loads[src] = max(0, loads[src] - transfer_amount)  # 避免
负数
        # 向目的地分拣中心添加货量
        loads[dst] += transfer_amount
    # 更新DataFrame
    for center, new_load in loads.items():
        updated_loads.loc[(updated_loads['日期'] == date) &
        (updated_loads['分拣中心'] == center), '货量'] = new_load
updated_loads.to_csv('answer2_1.csv',index=False)
avg_distribution = pd.read_csv('avg_distribution.csv')
    def distribute_daily_to_hourly(daily_forecast,
    hourly_distribution):
    forecast_detailed = daily_forecast.merge(hourly_distribution
    , on=['分拣中心'])
    forecast_detailed['预测货量'] = forecast_detailed['货量']
    * forecast_detailed['小时比例']
    return forecast_detailed[['分拣中心', '日期', '小时',
    '预测货量']]
result_hourly = distribute_daily_to_hourly
(updated_loads, avg_distribution)
result_hourly.to_csv('answer2_2.csv',index=False)
```

问题三：python源代码

```
import numpy as np
import pandas as pd
import pulp as pp
```

```
from tqdm import tqdm
hourly_prediction = pd.read_csv('answer2_2.csv')
def solve_optimization(center, date,example_data):
    x = {i: pp.LpVariable(f正式工班次"{i}", lowBound=0,
        cat="Integer") for i in range(1, 7)}
    y = {i: pp.LpVariable(f临时工班次"{i}", lowBound=0,
        cat="Integer") for i in range(1, 7)}
    mylp = pp.LpProblem("lp1",pp.LpMinimize)
    objective_func = x[1] + x[2] + x[3] + x[4] + x[5] +
    x[6] + y[1] + y[2] + y[3] + y[4] + y[5] + y[6]
    mylp += objective_func
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[0]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[1]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[2]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[3]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[4]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] + 25*x[2] + 20*y[2] >=
    example_data.iloc[5]['预测货量'])
    mylp += (25*x[1] + 20*y[1] + 25*x[2] + 20*y[2] >=
    example_data.iloc[6]['预测货量'])
    mylp += (25*x[1] + 20*y[1] + 25*x[2] + 20*y[2] >=
    example_data.iloc[7]['预测货量'])
    mylp += (25*x[2] + 20*y[2] + 25*x[3] + 20*y[3] >=
    example_data.iloc[8]['预测货量'])
    mylp += (25*x[2] + 20*y[2] + 25*x[3] + 20*y[3] >=
    example_data.iloc[9]['预测货量'])
    mylp += (25*x[2] + 20*y[2] + 25*x[3] + 20*y[3] >=
    example_data.iloc[10]['预测货量'])
    mylp += (25*x[2] + 20*y[2] + 25*x[3] + 20*y[3] >=
    example_data.iloc[11]['预测货量'])
    mylp += (25*x[3] + 20*y[3] + 25*x[4] + 20*y[4] >=
    example_data.iloc[12]['预测货量'])
    mylp += (25*x[3] + 20*y[3] + 25*x[4] + 20*y[4] >=
    example_data.iloc[13]['预测货量'])
```

```
    mylp += (25*x[3] + 20*y[3] + 25*x[4] + 20*y[4] + 25*x[5] +
    20*y[5] >= example_data.iloc[14]['预测货量'])
    mylp += (25*x[3] + 20*y[3] + 25*x[4] + 20*y[4] + 25*x[5] +
    20*y[5] >= example_data.iloc[15]['预测货量'])
    mylp += (25*x[4] + 20*y[4] + 25*x[5] + 20*y[5] + 25*x[6] +
    20*y[6] >= example_data.iloc[16]['预测货量'])
    mylp += (25*x[4] + 20*y[4] + 25*x[5] + 20*y[5] + 25*x[6] +
    20*y[6]>= example_data.iloc[17]['预测货量'])
    mylp += (25*x[4] + 20*y[4] + 25*x[5] + 20*y[5] + 25*x[6] +
    20*y[6]>= example_data.iloc[18]['预测货量'])
    mylp += (25*x[4] + 20*y[4] + 25*x[5] + 20*y[5] + 25*x[6] +
    20*y[6]>= example_data.iloc[19]['预测货量'])
    mylp += (25*x[5] + 20*y[5] + 25*x[6] + 20*y[6]>=
    example_data.iloc[20]['预测货量'])
    mylp += (25*x[5] + 20*y[5] + 25*x[6] + 20*y[6]>=
    example_data.iloc[21]['预测货量'])
    mylp += (25*x[6] + 20*y[6]>= example_data.iloc[22]
    ['预测货量'])
    mylp += (25*x[6] + 20*y[6]>= example_data.iloc[23]
    ['预测货量'])
    mylp += (x[1] + x[2] + x[3] + x[4] + x[5] + x[6] <= 60)
    mylp.solve(pp.PULP_CBC_CMD(msg=0)) 求解函数#
    # 定义班次时间
    shifts = {
        1: '00:00-08:00',
        2: '05:00-13:00',
        3: '08:00-16:00',
        4: '12:00-20:00',
        5: '14:00-22:00',
        6: '16:00-24:00'
    }
    result_list = []
    for i in range(1,7):
        result_dict = {}
        result_dict['分拣中心'] = center
        result_dict['日期'] = date
        result_dict['班次'] = shifts[i]
        result_dict['正式工人数'] = x[i].varValue
        result_dict['临时工人数'] = y[i].varValue
```

```
        result_list.append(result_dict)
    return result_list
# 调用函数并汇总结果
all_results = []
problem_res = []
# 假设已经加载为hourly_predictionDataFrame
# 现在执行你的代码
centers = hourly_prediction['分拣中心'].unique()
dates = hourly_prediction['日期'].unique()
for center in tqdm(centers):
    print(center)
    for date in dates:
        print(date)
        example_data = hourly_prediction[hourly_prediction
        ['分拣中心']==center]
        example_data['日期'] = example_data['日期'].apply(str)
        example_data = example_data[example_data['日期']==date]
        results = solve_optimization(center, date,example_data)
        if results:
            all_results.extend(results)
        else:
            print(center,date)
            problem_res.append((center,date))
res = pd.DataFrame(all_results)
res.to_csv('answer3_2.csv',index=False)
problem_res = pd.DataFrame(problem_res)
problem_res.to_csv('problem.csv',index=False)
```

问题四：python源代码

```
# 导入所需要的资源包
import numpy as np
import pandas as pd
import pulp as pp
from tqdm import tqdm
# 存储正式工的休息日
rest_days = {i: pp.LpVariable(f正式工休息日"{i}", cat="Binary")
    for i in range(1, 7)}
# 导入问题二中预测的结果
hourly_prediction = pd.read_csv('D:\Program\python\jianmo结果表\4.csv')
```

```
# 解决工人的排班问题
def solve_optimization(center, date, example_data):
    # 正式工的班次
    x = {i: pp.LpVariable(f正式工班次"{i}", lowBound=0,
        cat="Integer")
        for i in range(1, 7)}
    # 临时工的班次
    y = {i: pp.LpVariable(f临时工班次"{i}", lowBound=0,
        cat="Integer")
        for i in range(1, 7)}
    mylp = pp.LpProblem("lp1", pp.LpMinimize)
    # 将正式工和临时工的工作时间分为六个班次，通过，两个数组存储xy
    objective_func = x[1] + x[2] + x[3] + x[4] + x[5] + x[6];
    for i in range(1, 3):
        mylp += (rest_days[i] + rest_days[i + 1] + rest_days
        [i +2] + rest_days[i + 3] + rest_days[i + 4] >= 1)
    mylp += objective_func
    # 保证总工作量不能低于预测货量
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[0]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[1]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[2]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[3]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] >= example_data.iloc[4]
    ['预测货量'])
    mylp += (25*x[1] + 20*y[1] + 25*x[2] + 20 *
    y[2] >= example_data.iloc[5]['预测货量'])
    mylp += (25*x[1] + 20*y[1] + 25*x[2] + 20 *
    y[2] >= example_data.iloc[6]['预测货量'])
    mylp += (25*x[1] + 20*y[1] + 25*x[2] + 20 *
    y[2] >= example_data.iloc[7]['预测货量'])
    mylp += (25*x[2] + 20*y[2] + 25*x[3] + 20 *
    y[3] >= example_data.iloc[8]['预测货量'])
    mylp += (25*x[2] + 20*y[2] + 25*x[3] + 20 *
    y[3] >= example_data.iloc[9]['预测货量'])
    mylp += (25*x[2] + 20*y[2] + 25*x[3] + 20 *
```

```
    y[3] >= example_data.iloc[10]['预测货量'])
    mylp += (25*x[2] + 20*y[2] + 25*x[3] + 20 *
    y[3] >= example_data.iloc[11]['预测货量'])
    mylp += (25*x[3] + 20*y[3] + 25*x[4] + 20 *
    y[4] >= example_data.iloc[12]['预测货量'])
    mylp += (25*x[3] + 20*y[3] + 25*x[4] + 20 *
    y[4] >= example_data.iloc[13]['预测货量'])
    mylp += (25*x[3] + 20*y[3] + 25*x[4] + 20*y[4] + 25 *
    x[5] + 20*y[5] >= example_data.iloc[14]['预测货量'])
    mylp += (25*x[3] + 20*y[3] + 25*x[4] + 20*y[4] + 25 *
    x[5] + 20*y[5] >= example_data.iloc[15]['预测货量'])
    mylp += (25*x[4] + 20*y[4] + 25*x[5] + 20*y[5] + 25 *
    x[6] + 20*y[6] >= example_data.iloc[16]['预测货量'])
    mylp += (25*x[4] + 20*y[4] + 25*x[5] + 20*y[5] + 25 *
    x[6] + 20*y[6] >= example_data.iloc[17]['预测货量'])
    mylp += (25*x[4] + 20*y[4] + 25*x[5] + 20*y[5] + 25 *
    x[6] + 20*y[6] >= example_data.iloc[18]['预测货量'])
    mylp += (25*x[4] + 20*y[4] + 25*x[5] + 20*y[5] + 25 *
    x[6] + 20*y[6] >= example_data.iloc[19]['预测货量'])
    mylp += (25*x[5] + 20*y[5] + 25*x[6] + 20 *
    y[6] >= example_data.iloc[20]['预测货量'])
    mylp += (25*x[5] + 20*y[5] + 25*x[6] + 20 *
    y[6] >= example_data.iloc[21]['预测货量'])
    mylp += (25*x[6] + 20*y[6] >= example_data.iloc[22]
    ['预测货量'])
    mylp += (25*x[6] + 20*y[6] >= example_data.iloc[23]
    ['预测货量'])
    # 正式工的出勤人数不能超过人200
    mylp += (x[1] + x[2] + x[3] + x[4] + x[5] + x[6] <= 200)
    # 将正式工的休息日变量添加到目标函数中
    objective_func += rest_days[1] + rest_days[2] + \
    rest_days[3] + rest_days[4] + rest_days[5] + rest_days[6]
    mylp.solve(pp.PULP_CBC_CMD(msg=0))  # 求解函数
    # 定义班次时间
    shifts = {
        1: '00:00-08:00',
        2: '05:00-13:00',
        3: '08:00-16:00',
        4: '12:00-20:00',
```

```
            5: '14:00-22:00',
            6: '16:00-24:00'
        }
        # 建立结果表格
        result_list = []
        for i in range(1, 7):
        result_dict = {}
        result_dict['分拣中心'] = center
        result_dict['日期'] = date
        result_dict['班次'] = shifts[i]
        result_dict['正式工人数'] = x[i].varValue
        result_dict['临时工人数'] = y[i].varValue
        result_list.append(result_dict)
        return result_list
# 调用函数并汇总结果
all_results = []
problem_res = []
# 假设已经加载为hourly_predictionDataFrame
# 现在执行你的代码
centers = hourly_prediction['分拣中心'].unique()
dates = hourly_prediction['日期'].unique()
for center in tqdm(centers):
    print(center)
    for date in dates:
        print(date)
        example_data = hourly_prediction[hourly_prediction
        ['分拣中心'] == center]
        # example_data['日期'] = example_data['日期'].apply(str)
        example_data.loc[:, '日期'] = example_data['日期'].
        apply(str)
        example_data = example_data[example_data['日期'] == date]
        results = solve_optimization(center, date, example_data)
        if results:
            all_results.extend(results)
        else:
            print(center, date)
            problem_res.append((center, date))
# 将结果存入结果表中5
res = pd.DataFrame(all_results)
```

```
128 res.to_csv('结果表5.csv', index=False)
129 problem_res = pd.DataFrame(problem_res)
130 # 将错误的数据存入中problem
131 problem_res.to_csv('problem.csv', index=False)
```